

Applied Social Data Science

Coding Camp

Setup • Orientation

Elena Karagianni

karagiae@tcd.ie

 September 7–11, 2026

Department of Political Science, Trinity College Dublin

Schedule

10am - 1pm

- Monday: Introduction + R Basics I
- Tuesday: R Basics II + Good practices

10am - 12pm

- **Thursday: Python basics**
- Friday: How to write up and report on $\text{\LaTeX}$

From R to Python

- Python is a **general-purpose** language; R is a **statistical** language
- Python dominates machine learning, deep learning, automation, and web/app development
- R is built *by* statisticians *for* data analysis; Python is built by developers for “almost anything”

From R to Python

- Python is a **general-purpose** language; R is a **statistical** language
- Python dominates machine learning, deep learning, automation, and web/app development
- R is built *by* statisticians *for* data analysis; Python is built by developers for “almost anything”
- Good news: most of what you learned Tuesday (objects, functions, logic) transfers directly

R vs. Python: The Essentials

Aspect	R	Python
Designed for	Statistics & analysis	General-purpose programming
Typical users	Researchers, statisticians	Developers, data/ML engineers
Assignment	<code><-</code> (or <code>=</code>)	<code>=</code> only
Indexing	Starts at 1	Starts at 0
Code blocks	<code>{ }</code> curly braces	Indentation (whitespace)
Packages	<code>install.packages()</code>	<code>pip install / conda install</code>
Loading a package	<code>library(pkg)</code>	<code>import pkg</code>

Same Logic, Different Syntax

R

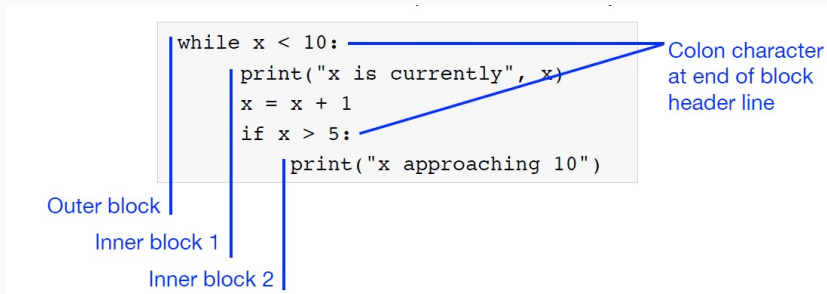
```
x <- 5
if (x > 3) {
  print("big")
}
```

Python

```
x = 5
if x > 3:
    print("big")
```

- No curly braces → the colon **:** and **indentation** define the block
- No semicolons needed at the end of lines

Indentation Matters



- In R, whitespace is cosmetic; in Python, it is **syntax**
- Use **4 spaces** per level, consistently (Jupyter does this for you)
- Inconsistent indentation → ***IndentationError***

Two Ways to Run Python

- **Interactive mode (REPL):** type one line, see the result immediately
 - Great for testing ideas and exploring data
 - This is exactly what a **Jupyter Notebook cell** does

Two Ways to Run Python

- **Interactive mode (REPL):** type one line, see the result immediately
 - Great for testing ideas and exploring data
 - This is exactly what a **Jupyter Notebook cell** does
- **Script mode:** write a full program in a file, run it all at once
 - File extension: *.py*
 - Run from the terminal: *python myscript.py*
 - Better for reproducible, shareable analysis pipelines

Two Ways to Run Python

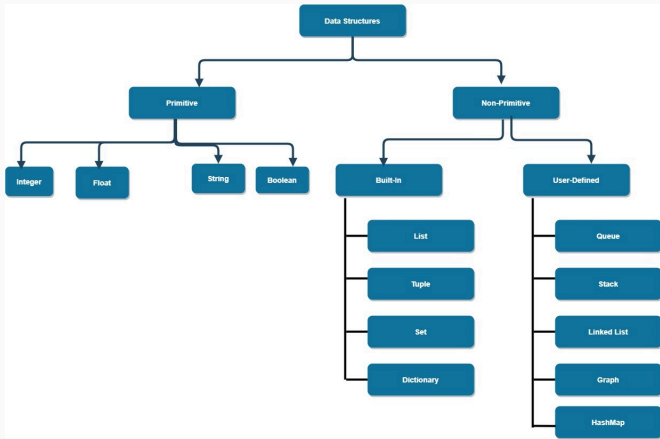
- **Interactive mode (REPL):** type one line, see the result immediately
 - Great for testing ideas and exploring data
 - This is exactly what a **Jupyter Notebook cell** does
- **Script mode:** write a full program in a file, run it all at once
 - File extension: *.py*
 - Run from the terminal: *python myscript.py*
 - Better for reproducible, shareable analysis pipelines
- Today we will use Jupyter Notebooks — interactive, but saved as a reproducible document

Python Data Types

Every variable in Python has a **type**, indicating the nature of the value it stores.

Python type	Example	R equivalent
<i>int</i>	5	<i>integer</i>
<i>float</i>	3.14	<i>numeric</i>
<i>str</i>	"hello"	<i>character</i>
<i>bool</i>	True / False	<i>logical (TRUE/FALSE)</i>
<i>NoneType</i>	None	<i>NULL</i>

Python Data Structures



- Today: the four **built-in** structures — *list*, *tuple*, *set*, *dict*

Lists: Ordered & Mutable

```
fruits = ["apple", "banana", "cherry"]  
fruits[0] # 'apple' <- Python starts at 0!  
fruits.append("date")  
len(fruits) # 4
```

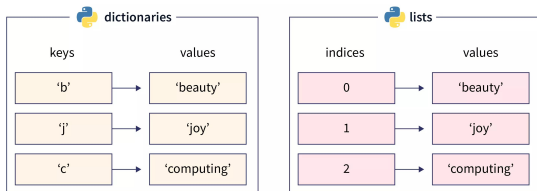
- Ordered, changeable (**mutable**), can hold mixed types
- Closest R equivalent: a plain `list()` or vector, but 0-indexed

Tuples: Ordered & Immutable

```
point = (10, 20)
point[0] # 10
point[0] = 99 # TypeError: tuples cannot be changed
```

- Same as a list, but written with `()` and **cannot be modified** after creation
- Use it for fixed data: coordinates, RGB colours, function returns

Dictionaries: Key-Value Pairs



SCALER
Topics

```
student = {"name": "Elena", "age": 25}  
student["name"] # 'Elena'
```

- Accessed by **key**, not position — like a **named list** in R

Sets: Unique & Unordered

```
nums = {1, 2, 2, 3}
nums # {1, 2, 3} duplicates removed
"apple" in fruits # membership test -> True/False
```

- No duplicates, no guaranteed order
- Handy for de-duplicating data and fast membership checks

Let's try it out!

Step 1

Open **Anaconda Prompt** first

Then: launch Jupyter Notebook from Anaconda Navigator